

PySpark Code Questions for Data Engineer (Interview Focused)



These are real interview—asked PySpark coding questions **with** clear answers and best practices, **suitable for**

1–5+ years of experience.

1. How do you read large files efficiently in PySpark?

Answer

```
df = spark.read \
    .option("header", "true") \
    .option("inferSchema", "false") \
    .csv("s3://data/input/")
```

Best Practices

- Avoid `inferSchema` for large datasets
- Define schema explicitly
- Prefer **Parquet** over CSV for performance

```
from pyspark.sql.types import *

schema = StructType([
    StructField("id", IntegerType(), True),
    StructField("name", StringType(), True)
])
```

2. Difference between `select()` and `withColumn()`?

Answer

```
df.select("id", "salary")
df.withColumn("bonus", col("salary") * 0.1)
```

Best Practices

- `select()` is faster for projections
- Avoid chaining multiple `withColumn()` calls

```
df.select(
    "id",
    (col("salary") * 0.1).alias("bonus")
)
```

3. How do you handle NULL values?

Answer

```
df.fillna({"salary": 0})
```

```
df.dropna(subset=["id"])
```

Best Practices

Use `coalesce()` for safe default handling:

```
from pyspark.sql.functions import coalesce, lit

df.withColumn("salary", coalesce(col("salary"),
lit(0)))
```

4. How do you remove duplicates?

Answer

```
df.dropDuplicates()
df.dropDuplicates(["id"])
```

Best Practices (Latest Record)

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

w =
Window.partitionBy("id").orderBy(col("updated_at").de
sc())
df.withColumn("rn", row_number().over(w)) \
    .filter(col("rn") == 1)
```

5. Difference between `groupBy()` and window functions?

Answer

```
df.groupBy("dept").sum("salary")
```

Window:

```
from pyspark.sql.window import Window
from pyspark.sql.functions import sum

w = Window.partitionBy("dept")
df.withColumn("dept_total", sum("salary").over(w))
```

Best Practices

- `groupBy()` reduces rows
 - Window functions keep row-level detail
-

6. How do you optimize joins?

Answer

```
from pyspark.sql.functions import broadcast

df.join(broadcast(dim_df), "id")
```

Best Practices

- Broadcast small dimension tables
- Avoid shuffle joins
- Choose correct join type
- Repartition when required

7. Difference between `repartition()` and `coalesce()`?

Answer

```
df.repartition(10)
df.coalesce(5)
```

Best Practices

- `repartition()` causes shuffle
 - `coalesce()` avoids shuffle
 - Use `coalesce()` before writing data
-

8. How do you handle data skew?

Answer

Use salting technique:

```
df.withColumn("salt", rand())
```

Best Practices

- Identify skewed keys using Spark UI
 - Broadcast small tables
 - Increase shuffle partitions
 - Use salting for heavy keys
-

9. Difference between `cache()` and `persist()`?

Answer

```
df.cache()  
df.persist(StorageLevel.MEMORY_AND_DISK)
```

Best Practices

- Use `cache()` when memory is sufficient
- Use `persist()` for large datasets
- Always unpersist after use

10. How do you write data efficiently?

Answer

```
df.write \  
  .mode("overwrite") \  
  .partitionBy("date") \  
  .parquet("s3://output/")
```

Best Practices

- Partition on commonly filtered columns
 - Avoid small files
 - Use Parquet or Delta
 - Use `coalesce()` before write
-

11. Difference between `DataFrame` and `RDD`?

Answer

- `DataFrame`: Optimized using Catalyst and Tungsten
- `RDD`: Low-level, no query optimization

Best Practices

Prefer DataFrames in almost all cases
Use RDDs only for low-level custom logic

12. How do you debug slow Spark jobs?

Answer

- Use Spark UI
- Analyze stages and shuffle operations

Best Practices

- Reduce shuffles
- Optimize joins
- Cache reused DataFrames
- Tune `spark.sql.shuffle.partitions`

Tech Interview Titans